What Does It Mean to Win? Metric Choice Reverses the Strategy Ranking in Rummikub

Abstract—We study which simple strategy wins at two-player Rummikub, and find that the answer depends on how winning is measured. In a simulator of the official rules, a paired round-robin of seven interpretable strategies is won by the *joker-saver*, which plays as many tiles as possible but spends a wildcard only to empty its hand. Four ablations ($13{,}000$ games) isolate the cause: forbidding the joker as the final tile reverses the advantage, and 71% of the saver's wins end with a joker. Official Rummikub, however, is zero-sum in tile points, with a joker left in hand counting 30. Scoring the same $18{,}000$ games both ways, the saver is first in games won and sixth of ten in points. Over sessions of ten hands with accumulated points, the format the rules define, the inversion is complete: the saver wins the most hands of any strategy and the fewest sessions, losing 72.5% of sessions to max-tiles, and the session ranking is the exact reverse of the per-hand win ranking. A per-hand win rate of 0.573 corresponded to losing three sessions in four. Twenty-one learned and search agents, including PPO, Deep Monte Carlo, a determinised search given the true hidden hands, and training on the official reward, fail to beat the best heuristic under either metric. Scaling the training budget $27\times$ to $576{,}000$ games produces the first learned agent to beat the champion in points ($+4.69$ $[+2.10, +7.28]$), and it does so by *winning fewer hands* and losing more cheaply; in a $9{,}360$-game common field it still only ties the best heuristic (-1.69 $[-3.70, +0.31]$ paired), so the best Rummikub player here remains a three-line rule.

Index Terms—Game AI, imperfect information, evaluation metrics, reinforcement learning, benchmark design, Rummikub.

1 Introduction

Rummikub is a tile game in which players empty their hands by forming runs and groups on a shared table. Its *optimisation* subproblems are well studied: forming the best set of melds from a hand is polynomial-time solvable but becomes hard under generalisations [1], and integer linear programming solves full table-rearrangement problems [2]. Much less is known about which *playing policies* win more often. Popular advice ("keep your jokers") is widespread but, to our knowledge, untested.

This paper reports a result that began as a strategy study and became a measurement study. We first confirm the folk advice in its conditional form and explain it mechanistically: among seven interpretable strategies, the one that hoards wildcards until they can end the game wins the most games, and an intervention on that single exception reverses the effect. We then observe that the quantity we had been maximising — games won — is not the quantity the rules define. Official Rummikub is zero-sum in tile points, a joker left in hand counts 30, and play is organised in sessions of several hands with accumulated scores. Re-scoring the same games, and then simulating sessions, reverses the ranking: the policy that wins the most hands wins the fewest sessions.

We ask four research questions.

- RQ1 Among a menu of interpretable strategies, which wins most often?
- RQ2 What *mechanism* explains the winner's advantage?
- RQ3 Does the answer survive the scoring metric printed in the rules, and the multi-hand format in which the game is played?
- RQ4 Can reinforcement learning (RL), a language-model decision-maker or a search agent beat the best fixed strategy, under either metric?

Status of the evidence: RQ1 is exploratory. The RQ2 experiments were designed after seeing the RQ1 result, with four competing explanations and a prediction for each, and are corrected as one multiple-comparison family; they were not pre-registered. The two RQ3 experiments and the official-reward run of RQ4 have development and test seeds with criteria fixed in writing before any data were collected, and the test seeds were spent once. All results are conditional on this simulator and this strategy menu. Supplementary material (anonymised repository) contains per-game records and every analysis script.

2 Related Work

Rummikub: The literature we found treats Rummikub as an optimisation problem: a polynomial-time algorithm for the one-hand puzzle [1], an integer-programming formulation for rearranging the table [2], a thesis counting winning hands [3], and one that reduces the solver's memory and classifies puzzle difficulty [4]. The last explicitly notes that the multi-player game (initial meld, table manipulation, strategy) has received little study and leaves strategy for future work. We found no study evaluating learning or search agents on the full multi-player game against heuristic baselines; this is the result of a limited search, not a proof of absence. The closest learning study we know is on a sibling game: temporal-difference learning with self-play lost to co-evolution of the value function in gin rummy [5].

Learning in large-action card games: DouZero reaches strong DouDizhu play by Monte Carlo returns, a network that scores encoded (state, action) pairs, and many

parallel actors, which suits action sets that change from turn to turn [6]; our move network likewise scores concrete moves rather than using a fixed action head. Suphx adds global reward prediction and oracle guiding to deep RL for mahjong [7]. Policy-gradient self-play against a pool of past checkpoints is common practice in tabletop-game RL [8], which our league runs follow. A large-scale study finds generic PPO [10] competitive with game-theoretic deep-RL methods in imperfect-information games [9]. Group-relative advantages, which replace a learned critic by standardising returns within a group of rollouts sharing a prompt, were introduced for language-model training [11]; we use the same estimator with a group of games sharing a deal. Our shaping term is a potential difference and therefore policy-invariant in the sense of [12]. The closest positive result to our own setup is IRumAI, which warm-starts PPO by behaviour cloning in Indian Rummy and reaches a 53.9% win rate against the strongest search baseline unseen in training [13]; our recipe is the same in outline (clone, then policy gradient) and does not reproduce that outcome, which we discuss in Section 9.

Search under hidden information: Perfect-information Monte Carlo (PIMC) sampling, which our search agent follows, is practical but suffers strategy fusion, since the search may choose different continuations in worlds the player cannot distinguish [14]; its empirical success has been characterised [15], and information-set MCTS searches the player's information set instead of sampled worlds [16].

Evaluation: Our methodological conclusion is adjacent to work arguing that deep-RL comparisons rely too heavily on point estimates from few runs [17]. That critique concerns *statistical* reliability at a fixed metric. We report a case where the metric itself, rather than its variance, reverses the conclusion.

3 Simulator and Strategies

Game: Two players in the main experiments; 106 tiles (4 colours $\times$ values 1–13 $\times$ 2 copies, plus 2 jokers); 14 tiles per hand. A turn either draws one tile or lays tiles down. The first lay-down (the *opening meld*) must be worth at least 30 points using only the player's own tiles and may not touch the table. Afterwards a player may lay several melds, extend melds, and rearrange the whole table, provided every meld is valid at the end of the turn and at least one tile from the hand was played. The two jokers are shuffled with the other tiles. A hand ends when a rack is empty; if the pool is empty and all players pass in succession, it ends as well. Seat 0 moves first. The engine supports two to ten players and was validated against an independent third-party solver (Section 8).

Scoring: Three rules appear in this paper. **Game count**: win $= 1$, draw $= 0.5$, loss $= 0$, the convention we started with. **Official points** [20]: the player who goes out scores the sum of all tiles left on opponents' racks, each loser scores minus its own rack, a joker counting 30; if the pool empties with nobody out, the lowest rack wins the differences. Both are zero-sum. **Session**: K hands, official points accumulated, highest total wins.

TABLE 1
The seven original strategies.

random	uniform over legal actions, drawing included
max-tiles	play the move using the most hand tiles (ties: larger tile sum); draw if none
max-points	play the move with the largest tile sum
min-play	play the smallest legal move
joker-saver	like max-tiles, but consider only moves that use no joker, *unless* some move empties the hand; otherwise draw
no-rearrange	like max-tiles but never modify melds already on the table
cautious	play only if the best move uses ≥ 4 tiles or goes out; otherwise draw

Action space: At each decision the simulator lists *draw*, *lay down new melds*, and *rearrange* actions. Because exhaustive table search is expensive, rearrangements are a sample of at most 128 plans; this is the main fidelity limitation (Sections 8 and 10).

Strategies: Each chooses one legal action. The original seven are in Table 1; nine more are introduced where they are first used.

4 Protocol and Statistics

Pairing: For every seed, all seatings are played with the same deal; the unit of analysis is the per-seed mean score of the strategy of interest. Pairing cancels the first-mover and deal effects: a strategy played against itself scores exactly 0.5 by construction, which gives a real control row rather than a nominal one. Seat 0 wins $50.8\% \pm 1.5\%$ of tournament games. In the session experiments the seed of hand k is derived from (session seed, k), so different strategies meet the same hands in the same position of a session, and the starting seat rotates each hand as the turn to start passes in the rules.

Intervals and tests: We report mean ± 1.96 standard errors across seeds. In the mechanism family (13 groups, two of them mirror controls with $p = 1$ by construction) p-values are Holm-corrected [18]; the confirmatory experiments of Sections 6.1 and 6.2 are Holm-corrected within their own families. Tournament intervals are descriptive. Where observations are clustered (two hands of a game, games of a seed) we use percentile bootstrap over seeds.

Scale: $123{,}000$ hands of two-player play across all experiments, of which $18{,}000$ are the dual-metric tournament and $36{,}000$ the session test; $36{,}000$ training games for the move network plus nineteen further training runs; three- to ten-player tables; and $2{,}100$ games against an external solver. Seed ranges are disjoint by experiment and listed in the supplement.

5 RQ1 and RQ2: Which Strategy, and Why

5.1 The heuristic tournament

Table 2 gives each strategy's mean score against the other six over $3{,}360$ paired games. The joker-saver leads. Max-tiles and max-points are statistically indistinguishable (head-to-head 0.509 ± 0.059); the saver beats max-tiles by $0.578 \pm$

TABLE 2
Round-robin score against the other six (95% CI across 80 seeds, both seats).

Strategy	Mean score
joker-saver	0.726 ± 0.027
max-tiles	0.676 ± 0.026
max-points	0.672 ± 0.022
min-play	0.555 ± 0.032
random	0.461 ± 0.026
no-rearrange	0.271 ± 0.019
cautious	0.140 ± 0.017

0.067. Two designs are clearly bad: never touching the table and drawing unless the move is large.

5.2 Mechanism

The joker-saver differs from max-tiles in exactly one respect, so the question is which part of that behaviour matters. We stated four candidate explanations and a prediction for each, then tested all four against max-tiles as opponent (500 seeds $\times$ 2 seats per row, Table 3, Fig. 1).

E1, a mild preference for spending fewer jokers, is not supported. Soft penalties of 0.5–4 tiles per joker score 0.500–0.508, none distinguishable from 0.5 after correction, and a joker *bonus* is identical to the control. The reason is visible in the telemetry: with soft penalties the player still spends 0.77 jokers per game, as often as the control (0.78), because a preference only breaks ties between moves of equal size. The saver's behaviour is qualitatively different: it *refuses* joker moves and draws instead, ending an average game with 0.31 jokers in hand.

E2, recycling jokers on the table, is not supported. When jokers can never be removed from table melds, the saver scores 0.581 ± 0.027, no lower than under standard rules (0.568 ± 0.027).

E3, timing, partly holds. Saving only before the opening gives nothing (0.495 ± 0.021); saving only after it gives 0.537 ± 0.022 ($p_{\text{Holm}} = 0.01$). The two timings do not add up to the full rule, so part of the effect requires both phases; we have not isolated it.

E4, the joker as a finishing tile, is supported. Forbidding the saver to use a joker even to empty its hand collapses it to 0.189 ± 0.021: it ends games holding 0.86 jokers and never goes out with one. Conversely, 71% of the full saver's wins end with a joker in the final move, against 4% for max-tiles. Because the two variants differ *only* in that exception, this is an intervention on the policy rather than a correlation.

So the folk advice is half right: unconditional saving is far worse than not saving, and the condition "unless it wins the game" carries the whole effect. A held wildcard keeps an awkward hand completable; this is our reading of E3–E4, and we did not test the narrower claim about awkward last tiles.

6 RQ3: The Metric and the Format

6.1 Official points

Everything above counts games. Official Rummikub counts tile points, and the joker-saver is precisely the policy for

TABLE 3
Mechanism experiments: score against max-tiles, Holm-adjusted p for mean = 0.5, jokers played per game, jokers left in hand at the end, and share of wins whose final move used a joker.

Variant	Score	p_{Holm}	Jok. left	Jok. finish
joker-saver (full rule)	0.568 ± 0.027	< 0.001	0.31	71%
jokers frozen	0.581 ± 0.027	< 0.001	0.32	75%
save after opening	0.537 ± 0.022	0.010	0.17	52%
save before opening	0.495 ± 0.021	1.000	0.01	5%
never play a joker	0.189 ± 0.021	< 0.001	0.86	0%
soft penalty $\lambda = 4$	0.508 ± 0.014	1.000	0.01	6%
soft penalty $\lambda = 2$	0.506 ± 0.006	0.514	0.01	6%
soft penalty $\lambda = 1$	0.503 ± 0.003	0.661	0.01	4%
soft penalty $\lambda = 0.5$	0.500 ± 0.000	1.000	0.01	4%
joker bonus $\lambda = -1$	0.500 ± 0.000	1.000	0.01	4%
joker bonus $\lambda = -2$	0.500 ± 0.000	1.000	0.01	4%
max-tiles mirror (control)	0.500 ± 0.000	1.000	0.01	4%

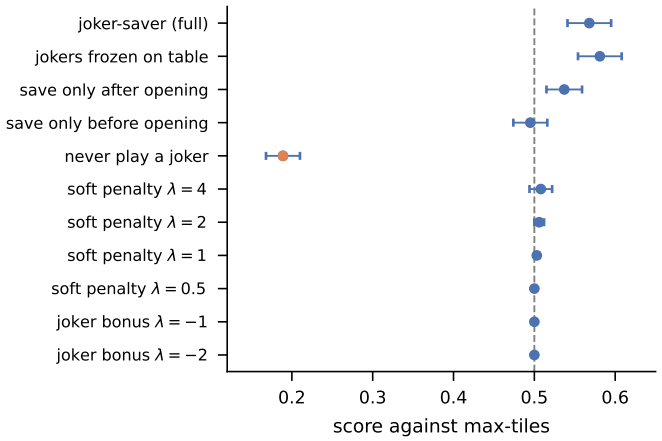


Fig. 1. Mechanism experiments (mean $\pm$ 95% CI against max-tiles, 500 seeds $\times$ 2 seats each). Only the full rule, its frozen-joker control and the after-opening variant beat 0.5; soft penalties and bonuses sit on the control, and removing the finishing exception (orange) reverses the effect.

which the two may disagree: it ends its losses holding 0.80 jokers, a 24-point liability per lost game. We therefore replayed the tournament computing both metrics *from the same games*, so they are perfectly paired, and added three strategies that decide when to release a joker, defined before running: *saver-then-dump* (the saver, but once the pool is empty it plays the largest move even with a joker), *saver-dump-near* (the saver, but it stops sparing jokers once an opponent holds three tiles or fewer) and *joker-spender* (prefers moves that spend jokers; a control for the opposite extreme). Criteria were fixed on development seeds (120 seeds); the test seeds (200 seeds $\times$ 2 seats, 18,000 games) were spent once with Holm over three hypotheses.

The ranking changes (Table 4, Fig. 2). The joker-saver is first by win score (0.680 ± 0.017) and sixth of ten by points ($+13.3 \pm 1.5$), below max-tiles and max-points ($+19.5$, $+19.6$), which it beats in games won. Spearman's correlation between the two orderings is 0.64, against 0.96 between alternative end-of-game rules (Section 8): the metric reorders the top far more than the rule does. Against a common field of five strategies, paired by seed, max-tiles

TABLE 4
Both metrics from the same 18,000 games (test seeds). Each cell is the mean against the other nine, ± 95% CI over seeds; * marks a strategy added here. Points are per game and zero-sum.

Strategy	Win score	Points	Pts rank	Jok. left
saver-then-dump*	0.689 ± 0.017	$+15.0 \pm 1.4$	5	0.19
joker-saver	0.680 ± 0.017	$+13.3 \pm 1.5$	6	0.27
max-points	0.639 ± 0.019	$+19.6 \pm 1.0$	2	0.00
max-tiles	0.633 ± 0.015	$+19.5 \pm 1.0$	3	0.00
joker-spender*	0.633 ± 0.015	$+19.5 \pm 1.0$	4	0.00
saver-dump-near*	0.622 ± 0.021	$+19.8 \pm 1.1$	1	0.07
min-play	0.495 ± 0.025	$+12.6 \pm 1.3$	7	0.00
random	0.440 ± 0.020	$+9.7 \pm 1.1$	8	0.00
cautious	0.160 ± 0.013	-26.7 ± 1.8	9	0.01
no-rearrange	0.009 ± 0.004	-102.3 ± 1.9	10	0.01

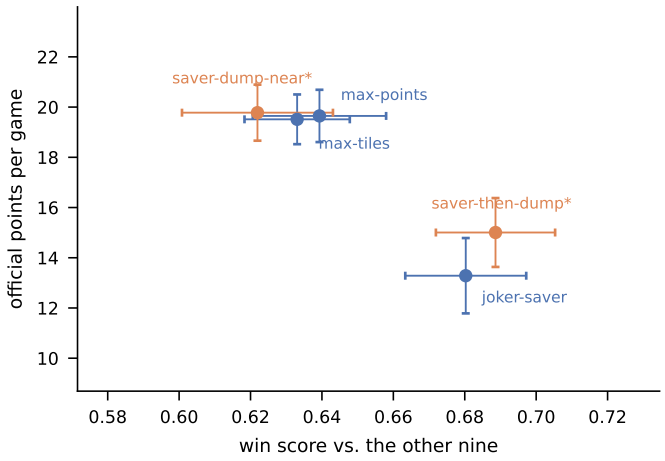


Fig. 2. The two metrics disagree, for the five strongest strategies by win score (orange: added in this section). The joker-saver is first in win score and last of the five in points. Joker-spender is omitted: it coincides with max-tiles in every comparable game.

is $+5.59$ $[+3.78, +7.39]$ points above the saver while being -0.013 ± 0.018 below it in win score (criterion $\geq +3.0$ with a positive interval; met, $p_{\text{Holm}} < 0.001$). Head-to-head the saver wins 0.573 ± 0.044 of games and still loses 5.4 ± 3.1 points per game.

The cause mirrors E4. Decomposing the saver's points against the common field: it wins 77.8% of those games, and when it wins it collects $+48.0$ points, the same as max-tiles ($+47.9$) — holding a joker buys the win, not a bigger win. When it loses it pays -43.3 against -14.6 for max-tiles, and 24 of those 29 extra points are the retained joker.

Intervening on the policy confirms the channel. *Saver-then-dump* ends its losses with 0.64 jokers instead of 0.80, gaining $+2.83$ $[+1.99, +3.67]$ points *and* $+0.019$ $[+0.012, +0.025]$ win score: releasing the joker once the pool is empty is free. The points gain fell short of the $+3.0$ fixed in advance, so we report it as a near miss. *Saver-dump-near* releases much earlier, ends with 0.09 jokers, pays -17.7 per loss, and is first of all ten in points ($+19.8 \pm 1.1$) while sixth in win score. *Joker-spender* is inert: under max-tiles play a joker is in hand in 2.6% of decisions and a choice between a joker move and a non-joker move of equal size arises in 0.1%, so it played identically to max-tiles in all 1,440 comparable games. That is a property of the action distribution, and it explains why the joker-bonus rows of Table 3 equal the control.

TABLE 5
Sessions of $K = 10$ hands (test seeds; 120 sessions per pair, both arrangements). Mean against the other five, ± 95% CI over sessions. *: not one of the original seven.

Strategy	Sessions won	Points per session	Hands won of 10
saver-dump-near*	0.655 ± 0.028	$+47 \pm 5$	4.76 ± 0.11
session-adaptive*	0.621 ± 0.028	$+18 \pm 7$	4.96 ± 0.09
max-tiles	0.560 ± 0.039	$+27 \pm 9$	4.77 ± 0.15
max-points	0.548 ± 0.041	$+24 \pm 9$	4.67 ± 0.15
saver-then-dump*	0.329 ± 0.030	-49 ± 9	5.43 ± 0.11
joker-saver	0.288 ± 0.030	-67 ± 10	5.39 ± 0.11

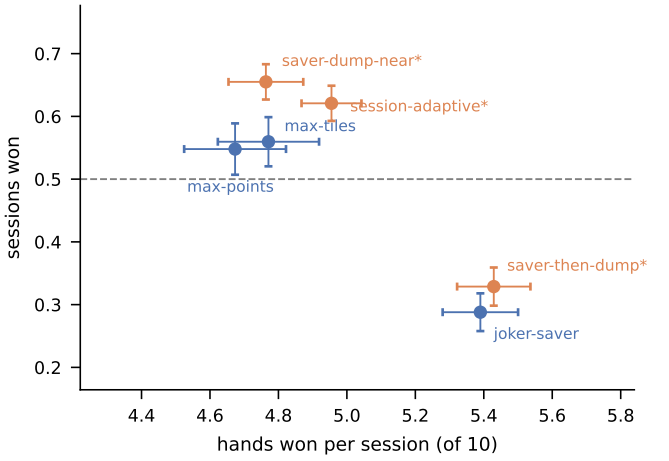


Fig. 3. Winning hands and winning sessions pull apart. The joker-saver wins the most hands per session of any strategy and the fewest sessions; releasing the wildcard before the end (orange) does the opposite.

6.2 Sessions of several hands

Rummikub is normally played as a session: several deals, points accumulated, highest total wins. Summing hands narrows the distribution around the mean, and the saver's mean is negative, so the session should amplify the effect. A session also creates information no single-hand strategy has, the accumulated balance, so we added *session-adaptive*: when behind by more than one joker (-30 points, fixed in advance) it plays the joker-saver, maximising its chance of winning the hand; otherwise it plays saver-dump-near. Six strategies, 15 pairs, $K = 10$, both seat arrangements; development seeds (80 sessions per pair) then test seeds once (120 sessions per pair, 3,600 sessions, 36,000 hands), Holm over two primary criteria.

The inversion is complete (Table 5, Fig. 3). The joker-saver wins 5.39 of the 10 hands in a session, more than any other strategy, and 0.288 ± 0.030 of the sessions, fewer than any other strategy. Head-to-head against max-tiles it takes 5.53 hands of 10 and loses 72.5% $[66.8, 78.2]$ of the sessions (criterion ≥ 0.55 with the interval above 0.5; met, $p_{\text{Holm}} < 0.001$) at -75.6 ± 14.9 points per session. The session ranking matches the per-hand *points* ranking

(Spearman $+0.90$) and is the exact reverse of the per-hand *win* ranking (Spearman -1.00) on the five strategies shared by both experiments. The effect grows with session length, as the arithmetic predicts: max-tiles beats the saver in 0.558 ± 0.089 of one-hand sessions, 0.754 ± 0.077 at $K = 5$ and 0.754 ± 0.077 at $K = 20$, with the points margin rising from $+14$ to $+142$.

Session-adaptive failed, in the informative direction. It loses to saver-dump-near, the fixed strategy it switches away from (0.444 $[0.414, 0.473]$; criterion not met, and the deficit itself is significant, $p_{\text{Holm}} = 0.0002$). The rule is not inert: it is behind the threshold in 26% of hands. Switching to joker-saving when behind buys hands (4.96 per session, the most of the four strong strategies) and costs points. Chasing a deficit by saving jokers digs the hole deeper; the accumulated balance is real information that this rule turns into a loss, and we report it rather than tuning a threshold fixed in advance at one value.

7 RQ4: Learned and Searching Agents

We ran twenty-one agents that choose either a strategy from the menu or a concrete move (Table 6). None beats the best heuristic. Rather than narrate each, we state what the table shows and then the four results that carry information beyond "no gain": what the architecture can and cannot find, an estimate that did not replicate, the two excuses that fail, and what $27\times$ more compute buys.

The architecture can represent the mechanism; the search does not find it: The move network scores each legal move from 15 features (move size, jokers used, hand and pool sizes, opening status, and so on) through one hidden layer of 16 tanh units, trained by REINFORCE against a league of heuristics, itself, and past checkpoints. Trained from scratch over $36{,}000$ games it converges to max-tiles-like play: 0.493 ± 0.029 against max-tiles, 0.403 ± 0.030 against the saver, going out with a joker in 2.9% of games against the saver's 41%. Supervised cloning of the saver on $10{,}587$ of its decisions reaches 82% agreement on held-out decisions and reproduces the mechanism (70% of its wins end with a joker), which shows the features suffice to express the rule. What the from-scratch run lacked was the search for it. The finishing-joker rule is a rare, delayed event concentrated in the last turns of a hand, which is the obvious candidate explanation for the policy-gradient failure at this budget; we did not test that explanation directly.

One positive estimate did not replicate: Cloning the saver and then refining it produced, on the first training seed, 0.605 against max-tiles — the only number in this project that appeared to beat both heuristics. Replicating with two further training seeds and evaluating on fresh seeds gave 0.509 ± 0.045: a favourable draw of seeds and run, selected by looking. Adding group-relative advantages [11] with four games per deal reaches 0.550 ± 0.040 against max-tiles ($p = 0.014$ uncorrected), the only interval in the paper excluding 0.5 against max-tiles, and we read it cautiously: it is one of four comparisons, and its difference from the moving-average baseline on the same seeds is not itself significant. Grouping does stabilise the mechanism, with 59–66% of wins ending with a joker across all three trainings against 14–43% without it. PPO showed the same pattern at a larger scale, scoring 0.500 against the saver on development seeds and 0.447 on held-out seeds used once.

Neither hidden information nor the reward is the binding constraint: Two experiments test the two most plausible excuses. Giving the determinised search the *true* world — the opponent's hand and the draw order — instead of sampled ones leaves it at 0.508 ± 0.089 against the saver, so perfect information about the hidden state does not lift this one-step search above the heuristic. And since Section 6.1 shows game count is the secondary metric, we retrained with reward $=$ points$/60$ from the same clones. The networks learn exactly what the heuristics say is worth points: they end a hand holding 0.22 jokers against the control's 0.28, go out with a joker in 58% of wins against 69%, and pay -33.2 points per loss against -38.7. But they gain only $+1.7$ $[-0.7, +4.1]$ points over the control, short of the $+3$ fixed in advance, and remain 5.1 points *below* max-tiles on the metric they optimise. The one behaviour the official reward teaches is a behaviour we can also write in one line of heuristic, where it is worth 4.5 points. What limits these agents is neither the information nor which number we reward.

Scaling the budget $27\times$ changes the mechanism, not the ranking: Every row of Table 6 trains on $7{,}200$ games, and we had declared that a limitation without measuring it. We therefore repeated the best recipe changing *only* the budget: $192{,}000$ games per seed, three seeds, $576{,}000$ games, 9.9 h on six cores, criteria fixed in advance, held-out seeds reserved. Against the joker-saver the scaled agent scores $+4.69$ $[+2.10, +7.28]$ official points, the first learned agent here whose interval excludes zero against the champion on the metric the rules define. Its isolated gain over the same recipe at the old budget, $+2.83$ $[-0.84, +6.51]$, falls short of the $+5$ we had required.

The mechanism inverts the usual story and is the reason we report this at all: the scaled agent *wins fewer hands* than its own control (0.390 against 0.487) yet scores more points, because it loses far more cheaply (-18.1 against -42.5 per loss). In defeat it ends with 2.78 tiles and 0.011 jokers in hand, against 4.31 and 0.604 for the control: it learned to release the joker and empty its hand *before* losing rather than chase the win. That is precisely the policy Section 6.1 says the official metric rewards, found by search rather than written by us, and no heuristic on our menu implements it. The budget also stopped binding early — training return saturated near iteration 450 of $4{,}000$, and the last $167{,}000$ games bought nothing — which removes budget from the list of explanations for the null results, alongside capacity, information and reward.

Is it then the best player? No: it ties with the best. A duel against two opponents cannot settle that, so we entered the three networks into the common-field tournament of Section 6.1: thirteen competitors, every pair, the same hands, both seats, $9{,}360$ games, seeds $270{,}000$+. The network ranks *first* in official points, $+16.96 \pm 1.38$ against max-tiles' $+15.60 \pm 1.44$ — and that ranking is an artefact. The paired comparison on identical hands against the same nine common opponents gives -1.69 $[-3.70, +0.31]$: the interval includes zero and the sign is negative. Head-to-head the network scores -0.60 $[-2.75, +1.55]$ points and wins 0.492 of hands, a technical tie, and in win rate it is sixth of eleven.

TABLE 6
Learned and searching agents, scored by game count against the two strongest heuristics ($\pm$ 95% CI; seeds are per opponent, both seats). Rows marked with a signed value are paired differences against the same architecture without the change. The last row is in official points. Every row other than the two PPO rows and the official-reward row is exploratory; multiplicity is not corrected across rows.

Agent	Variant	vs max-tiles	vs saver	Seeds	Status
Choosing a strategy from the menu					
Q-learner	tabular, 6,000 training games	0.410 ± 0.066	0.370 ± 0.060	100	below both
LLM	strategy selector, 600 games	0.500	0.400 ± 0.060	100	plays as max-tiles
Choosing a concrete move					
LLM	all legal moves listed	0.350 ± 0.076	0.414 ± 0.072	70	below both
PIMC	D=4 sampled worlds	0.542 ± 0.091	0.433 ± 0.085	60	below saver
PIMC	*true* world revealed	0.508 ± 0.089	0.508 ± 0.089	60	below saver
Network	REINFORCE from scratch	0.493 ± 0.029	0.403 ± 0.030	500	max-tiles-like
Network	clone of saver, untrained	0.525 ± 0.069	0.455 ± 0.069	100	below saver
Network	clone + REINFORCE	0.509 ± 0.045	0.487 ± 0.045	240	level with both
Network	clone + group adv. K=4	$\mathbf{0.550 \pm 0.040}$	0.495 ± 0.040	300	level with both
Network	+ hand-size shaping	0.505 ± 0.039	0.468 ± 0.041	300	no gain
Network	+ nine-opponent league	0.546 ± 0.061	0.500 ± 0.064	40	no gain
Network	+ rearrangement in training	0.535 ± 0.069	0.510 ± 0.069	100	no gain
Network	residual over the saver	0.460 ± 0.070	0.515 ± 0.070	100	never departs
Network	Deep Monte Carlo	0.555	0.472 ± 0.068	100	below saver
Network	PPO (development)	0.568	0.500	100	level, not better
Network	PPO (held out, used once)	0.560 ± 0.056	0.447 ± 0.056	150	did not replicate
Network	64 hidden units	-0.088 ± 0.048	–	100	worse than 16
Network	256 hidden units	$+0.012 \pm 0.047$	–	100	no gain
Network	19 opponent features	-0.010 ± 0.021	–	120	no gain
Network	23 tile-counting features	-0.007 ± 0.037	–	120	no gain
Network	reward = official points	-5.1 pts	–	100	below max-tiles

What survives is narrower: it beats the joker-saver in points head-to-head, $+5.59$ $[+1.42, +9.75]$, the only learned agent here to do so. The rank-order lead comes from each agent facing a slightly different field, which is why we report the paired difference rather than the ranking.

A language model as decision-maker: We also used a commercial System One model as a comparator, in two roles. As a strategy selector over 600 games it chose max-tiles in 84% of answered decisions and "draw" in 16%, never selecting the joker-saver, and scored 0.500 against max-tiles: in this protocol it *is* max-tiles. Given instead every legal move of the turn, with each option's concrete consequence computed in code (median 30 options), it scored 0.350 ± 0.076 against max-tiles, playing a joker in 3–4% of its moves. Its prompt stated the rules but no strategy advice, and in particular never described the finishing-joker mechanism; one prompt was used and was not tuned on results. Total spend was US$3.1.

8 Robustness

Five checks support the game-count results of Sections 5.2 and 6.1; all are reported in full in the supplement.

End-of-game rule. Our main experiments score a stalled pool as a draw (16.3% of games). Replaying the same 3,360 games under the official tie-break and under fewest-tiles leaves the top of the ranking and the mechanism unchanged (Spearman 0.96 for both; the saver's lead grows to 0.755 and the finishing-tile ablation stays at 0.164). The only reordering is at the bottom, where no-rearrange falls from 0.271 to 0.010: 54% of its games ended in pool exhaustion, so the draw rule had been flattering the weakest strategy.

Rearrangement cap. The 128-plan cap is reached in 9.9% of decisions and 73% of games, so it is not harmless. Replaying with caps of 512, 2,048 and 4,096 plans leaves the ranking identical in all four columns and *increases* the saver's head-to-head edge ($0.525 \rightarrow 0.625$), so truncation understates rather than creates the effect. An offline oracle re-enumerating 383 capped states found a play with more tiles in 4.2% of them, always by exactly one tile.

A wider menu. A 12-strategy league (66 pairs, 10,560 games) keeps the saver first by mean score and Bradley–Terry strength, though the three variants that keep a joker to the end are not separable from each other. No pair forms a three-way cycle, so strength is transitive on this menu. The finishing-joker pattern reappears: 59–62% of the wins of the saver and its closest variant end with a joker, against 0–18% for the others.

More players. At three players the saver keeps a smaller edge (0.374 against chance 0.333); at four it is exactly at chance (0.249), so the advantage vanishes as hands get shorter. The finishing-joker ablation still collapses at three players (0.159), so the mechanism holds where the advantage exists. Networks retrained with a curriculum over tables of two to ten players gain $+0.04$ to $+0.06$ at four, six and ten players on held-out seeds, and a network trained *only* on six-to-ten-player tables is indistinguishable from the curriculum one, so what matters is having seen large tables rather than the ones measured.

An independent opponent. All matches above are between policies we wrote, so we also played against third-party code: an open-source Rummikub solver [19] built on the integer-programming formulation of [2], whose default rule set matches ours exactly, confirming that our engine is

standard Rummikub. Wrapped in the obvious greedy player (ask for the best placement; play it if it melds anything, otherwise draw) it wins 0.040 $[0.015, 0.064]$ of two-player games against the saver and essentially none in three- and four-player fields. Optimal per-turn placement is not a policy: the solver melds 13 tiles per game and draws about 40 times, where the saver melds 21. Our trained networks beat it in essentially every game. This measures the solver outside its intended use, as a one-turn advisor to a human, but it does establish that our strategies are strong against an opponent we did not write.

9 Discussion

On Rummikub: The simulation supports a narrow, mechanistic answer to what wins a single hand by game count: play the biggest legal move, but keep jokers until one lets you go out. Three competing explanations fail and the finishing-tile intervention both removes and reverses the effect. But the practical advice inverts with the format. Under the scoring printed in the rules, keeping a joker costs about six points a hand against simply playing the most tiles, and over a session the same policy wins the most hands and the fewest sessions. The mechanism is one object seen from two sides: a held wildcard is a bet that pays a win and charges 30 points when it fails, and which side dominates is decided by the scoring rule and the format, not by the game. Releasing the joker before the end improves points and sessions at once, so the frontier is not a strict trade-off everywhere.

On measurement: The methodological point does not depend on Rummikub. Game count is the cheap, low-variance, conventional metric; points over a session is the one the rules define. On our menu they were *anti-correlated*: a per-hand win rate of 0.573, the kind of margin this literature routinely reports as an improvement, corresponded to losing three sessions in four, and the session ranking was the exact reverse of the per-hand win ranking. No number of additional seeds on the cheap metric would have revealed this, which distinguishes the problem from the statistical-reliability concerns of [17]: more runs sharpen an estimate of the wrong quantity. The practical recommendation is modest but concrete: when a game defines a score, measure it, and when it is played in sessions, simulate sessions, even when a per-episode proxy is available and cheaper. Where the proxy and the defined objective disagree, report both.

On the learning results: Twenty-one agents fail to beat the best heuristic under both metrics, and the eliminations are the informative part: the limiting factor is not representational capacity (a clone of the saver reproduces the mechanism from the same features), not hidden information (a search given the true world does not improve), not the reward (training on official points changes behaviour correctly but far too little), and not the training budget ($27\times$ more games saturates after 13% of them). What remains is credit assignment for a rare event at the end of a hand, the strength of the heuristic baseline, and the design of the observation itself: every feature we give the network is an aggregate scalar, which discards the two relations that define the game, since adjacent numbers form runs and distinct colours of one number form groups. The contrast with IRumAI [13], which reaches 53.9% against a strong search baseline in Indian Rummy with the same clone-then-policy-gradient outline, points to scale and observation design rather than the recipe; their own ablation makes a meld-aware convolutional encoder their largest single contributor, worth 8–11 percentage points over a width-matched MLP. We make no prediction about whether it would clear our baseline, but it is where we would look next, and the budget result rules out simply spending more compute on the representation we have. A joker-aware state, an explanation of the mechanism in the language model's prompt, search with a different rollout policy, and reward that is dense only in the last turns of a hand are the natural next experiments.

10 Limitations

- **Menu.** "Best" means best of sixteen hand-written strategies; stronger policies may exist. The move network has a 16-unit hidden layer and hyper-parameters were not tuned; only one value each of the learning rate, group size, shaping weight and adaptive threshold was tried. The budget experiment scaled games, not capacity or hyper-parameters, so it bounds the effect of games alone; we did not test whether a meld-aware observation would change the picture.
- **Joker identity on the table.** Officially a melded joker must keep representing the same tile unless the player supplies that tile. Our engine stores a joker as a value-free tile and validates only the final sets, so a rearrangement can silently change what a table joker represents. Our frozen-joker ablation is stricter than the official rule and the default is more permissive, so the true rule lies between two conditions we measured but was not itself measured. Since every main result concerns jokers, this is the fidelity gap we would close first.
- **Scope of the metric results.** The dual-metric tournament and the sessions are two players only, and the sessions use $K = 10$ (with $K \in \{1, 5, 20\}$ in one cell) and model hands as independent apart from the accumulated balance; we do not model opponents adapting across hands. The mechanism ablations, the league and the larger tables were run on game count.
- **Statistics.** Normal-approximation intervals over seeds; the Holm families cover the mechanism table, the league, and each confirmatory experiment separately, not the paper as a whole. Table 6 is uncorrected across rows, and the three training seeds of each learned variant share deals, so their pooled intervals are optimistic in the sense of [17].
- **Simulation only.** No human players, and the language-model results are one prompt and one pinned model version.

11 Conclusion

Among seven interpretable Rummikub strategies, hoarding wildcards until one can end the hand wins the most hands (0.726 ± 0.027), and an intervention on that single exception

reverses the advantage ($0.568 \rightarrow 0.189$), with 71% of its wins ending with a joker. Scored by the tile points the rules define, the same policy falls to sixth of ten, 5.6 points per hand below simply playing the most tiles, because it ends its losses holding 0.80 jokers at 30 points each. Played as sessions of ten hands with accumulated points, it wins the most hands of any strategy and the fewest sessions, losing 72.5% of sessions to max-tiles, and the session ranking is the exact reverse of the per-hand win ranking. Twenty-one learned and searching agents, including PPO, Deep Monte Carlo, a search given the true hidden hands, training on the official reward, and a $27\times$ larger budget, fail to beat the best heuristic under either metric; the largest-budget agent is the first to beat the champion in points, and it gets there by winning fewer hands and losing more cheaply, yet in a $9{,}360$-game common field it only ties the strongest heuristic. The strategy result is specific to this simulator and menu. The measurement result is not: the conventional metric and the one the rules define were anti-correlated here, and only simulating the format the game is played in revealed it.

REPRODUCIBILITY

Code, per-game records for every table, and all analysis scripts are in the anonymised supplementary repository, https://anonymous.4open.science/r/rummikub. Every experiment is a subcommand of one entry point, and each confirmatory experiment fixed its success criterion and its seed ranges before running; seed ranges are disjoint and documented per experiment. The language-model API key is read from the environment and is not stored.

REFERENCES

[1] J. N. van Rijn, F. W. Takes, and J. K. Vis, "The complexity of Rummikub problems," *arXiv:1604.07553*, 2016, (BNAIC 2015).

[2] D. den Hertog and P. B. Hulshof, "Solving Rummikub problems by integer linear programming," *The Computer Journal*, vol. 49, no. 6, pp. 665–669, 2006.

[3] R. Feijen, "Counting winning hands in Rummikub," B.Sc. thesis, LIACS, Leiden University, 2023.

[4] M. van der Wekken, "Algorithms for Rummikub puzzles," B.Sc. thesis, LIACS, Leiden University, 2025.

[5] C. Kotnik and J. K. Kalita, "The significance of temporal-difference learning in self-play training: TD-Rummy versus EVO-Rummy," in *Proc. 20th Int. Conf. Machine Learning (ICML)*, 2003, pp. 369–375.

[6] D. Zha, J. Xie, W. Ma, S. Zhang, X. Lian, X. Hu, and J. Liu, "DouZero: Mastering DouDizhu with self-play deep reinforcement learning," in *Proc. ICML*, 2021.

[7] J. Li *et al.*, "Suphx: Mastering Mahjong with deep reinforcement learning," *arXiv:2003.13590*, 2020.

[8] M. Balla, G. E. M. Long, J. Goodman, R. D. Gaina, and D. Perez-Liebana, "PyTAG: Tabletop games for multi-agent reinforcement learning," *arXiv:2405.18123*, 2024.

[9] M. Rudolph *et al.*, "Reevaluating policy gradient methods for imperfect-information games," *arXiv:2502.08938*, 2025.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[11] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv:2402.03300*, 2024.

[12] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. ICML*, 1999, pp. 278–287.

[13] V. Mohan *et al.*, "IRumAI: Reinforcement learning for Indian Rummy," *arXiv:2606.21975*, 2026.

[14] I. Frank and D. Basin, "Search in games with incomplete information: A case study using bridge card play," *Artificial Intelligence*, vol. 100, no. 1–2, pp. 87–123, 1998.

[15] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the success of perfect information Monte Carlo sampling in game tree search," in *Proc. AAAI*, 2010.

[16] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information set Monte Carlo tree search," *IEEE Trans. Computational Intelligence and AI in Games*, vol. 4, pp. 120–143, 2012.

[17] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Proc. NeurIPS*, 2021.

[18] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, pp. 65–70, 1979.

[19] "Rummikub solver library," open-source software under the MIT License, 2025. Repository cited in the supplementary material to preserve anonymity.

[20] J. McLeod, "Rummikub," *Card Games website*, accessed Oct. 2026. [Online]. Available: https://www.pagat.com/rummy/rummikub.html